



BURSA ULUDAĞ ÜNİVERSİTESİ

2025-2026 Eğitim Öğretim Yılı Bahar Dönem

Veri Yapıları Projesi

Üniversite

Uludağ Üniversitesi

Bölüm

Bilgisayar Mühendisliği

İsim Numara

Eylem Cafcav 032490074

Zeynep Özer 032490068

Mehmet Alp Bilgin 032490077

Burak Şenol 032490072

Emre Korkut 032490069

1. Projenin Amacı ve Kapsamı

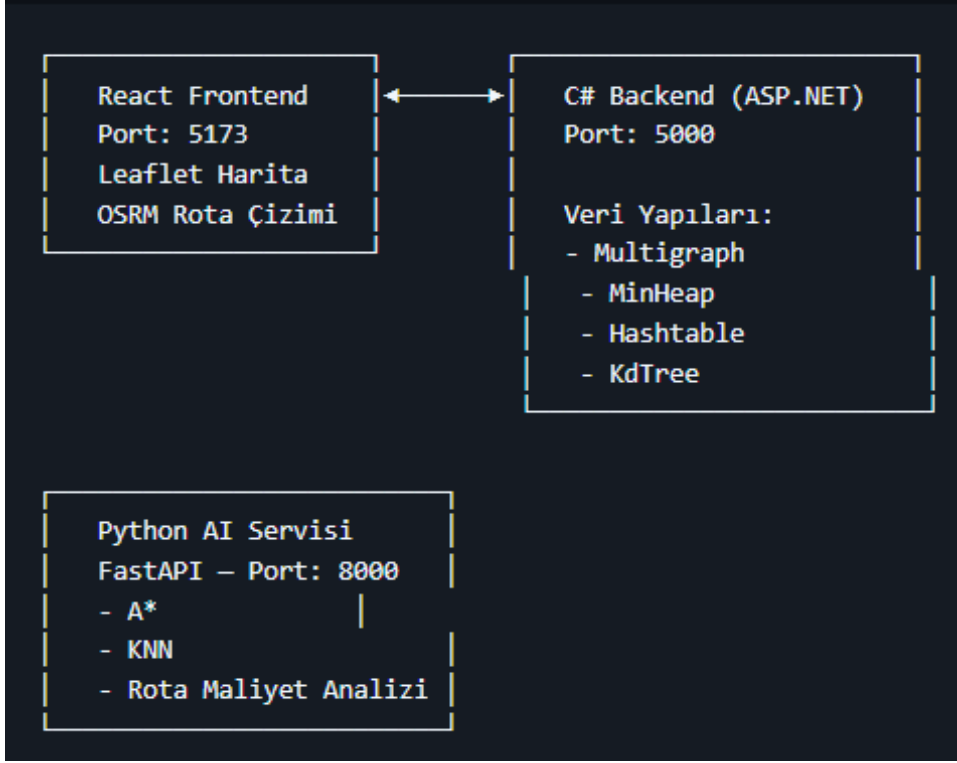
- Bu projenin temel amacı, karmaşık şehir içi toplu taşıma ağlarını matematiksel graf (çizge) yapıları ile modelleyerek, kullanıcılara A noktasından B noktasına en kısa, en az aktarmalı ve maliyet-etkin rotayı sunan akıllı bir navigasyon sistemi geliştirmektir. Proje, sadece çalışan bir algoritma değil; kullanıcı deneyimini merkeze alan modern bir arayüz, bağımsız çalışan mikroservisler ve sıfırdan inşa edilmiş temel veri yapıları (MinHeap, KdTree, Multigraph vb.) içermektedir.
- Ekip ve Görev Dağılımı:

1.	Eylem Cafcav	KdTree - Mekansal Arama (C#)
2.	Zeynep Özer	Multigraph, MinHeap, Hashtable - Çekirdek Veri Yapıları (C#)
3.	Mehmet Alp Bilgin	A*, KNN, Rota Analizi Algoritmaları (Python)
4.	Burak Şenol	Sentetik Veri Üretimi ve AI Servisi (Python/FastAPI)
5.	Emre Korkut	React GUI, Harita Entegrasyonu, Sistem Birleştirme (React/C#/Docker)

2. Mimari Tasarım ve Eşzamanlılık (Mikroservis Yaklaşımı)

- Projemiz, monolitik bir yapı yerine güncel endüstri standartlarına uygun olarak ayrı servisler halinde (Mikroservis Mimarisi) tasarlanmıştır.
- Eşzamanlılık ve Asenkron Yapı: Kullanıcı arayüzü (React/Vite), veri yönetimini ve köprü görevini üstlenen C# (ASP.NET) backend servisi ve ağır algoritmik hesaplamaları yapan Python (FastAPI) AI servisi birbirinden tamamen bağımsız portlarda (5173, 5000, 8000) çalışmaktadır.
- Thread-Safe Çalışma Ortamı: Kullanıcı harita üzerinde gezinirken veya rota sorgusu atarken, hesaplama motoru (Python) ayrı bir proseste asenkron olarak çalıştığı için arayüzde donma yaşanmaz. Aynı şekilde C#

backend servisimiz, gelen çoklu HTTP isteklerini kendi thread-pool yapısında izole ederek işler; böylece veri yapılarının tutulduğu ana bellekte eşzamanlılıktan kaynaklanan çakışmalar (race condition) önlenmiş ve thread-safe bir ortam sağlanmıştır.



(Sistem Mimarisinin Basitleştirilmiş bir diyagramı)

3. Kullanılan Veri Yapıları ve Zaman Karmaşıklığı (Big-O) Analizi

- Projede hazır kütüphanelere bağlı kalınmamış, algoritmaların bel kemiğini oluşturan veri yapıları sıfırdan (from scratch) kodlanmıştır.
- Multigraph Yapısı: Toplu taşıma ağı, düğümlerin (duraklar) ve kenarların (hatlar) bulunduğu bir Multigraph olarak modellenmiştir. Aynı iki durak arasından birden fazla hat geçebilmesi bu yapıyla sağlanmıştır. Durak ve hat ekleme işlemleri ortalama $O(1)$ sürede gerçekleşirken, komşu getirme maliyeti $O(E)$ 'dir. Uzay karmaşıklığı $O(V+E)$ olarak optimize edilmiştir.
- KdTree (K-Boyutlu Ağaç): Kullanıcının GPS konumuna (enlem/boylam) en yakın durağı bulmak için iki boyutlu (2D) uzaysal arama yapan

KdTree kullanılmıştır. N adet durak içinde en yakın komşuyu bulma işlemi, ardışık arama ($O(N)$) yerine $O(\log N)$ zaman karmaşıklığı ile çok daha performanslı hale getirilmiştir.

- Hashtable ($O(1)$ Durak Erişimi): Gelen rota isteklerinde durak verilerine anında erişebilmek için özel bir Hashtable yazılmıştır. id % 1009 hash fonksiyonu ve çakışma (collision) durumları için Linear Probing (Doğrusal Yoklama) yöntemi kullanılmıştır. Ortalama arama/ekleme süresi $O(1)$ 'dir.
- MinHeap (Öncelik Kuyruğu): A* algoritmasının açık listesini (open set) yönetmek için bir Min-Heap veri yapısı kodlanmıştır. Yeni bir rotayı kuyruğa ekleme (Insert) ve en düşük maliyetli rotayı çekme (RemoveMin) işlemleri $O(\log N)$ sürede tamamlanarak algoritmanın darboğaz yaşamayı engellenmiştir.

4. Algoritmalar ve Rota Maliyet Analizi

- Sistemdeki rota optimizasyonu, sezgisel (heuristic) yapısıyla bilinen A (A-Star) Algoritması* kullanılarak çözülmüştür.
- Maliyet Modeli: İki durak arasındaki maliyet sadece mesafe ile değil; "Ulaşım Süresi + Yürüyüş Süresi + (Aktarma Sayısı $\times$ 5 dakika)" formülüyle hesaplanmaktadır.
- Mesafe hesaplamalarında dünyanın küresel yapısını hesaba katan Haversine Formülü ($O(1)$) kullanılmıştır. A* algoritmasının zaman karmaşıklığı en kötü durumda $O((V + E) \log V)$ olarak gerçekleşmektedir. Ayrıca, KNN algoritması kullanılarak kullanıcının bulunduğu noktaya en yakın alternatif başlangıç durakları da hesaba katılmaktadır.

5. Versiyon Kontrolü, Konfigürasyon ve Teslimat Altyapısı

- Git ve İşbirliği: Geliştirme süreci tamamen Git üzerinden yürütülmüştür. Ana dallara (master/main) doğrudan kod itilmemiş (push), ekip üyeleri özellik bazlı dallar (branch) oluşturarak Pull Request (PR) mekanizması ile kod incelemeleri yapmıştır.
- Docker Konfigürasyonu: Tüm bileşenlerin (Frontend, Backend, AI Servisi) herhangi bir bağımlılık çatışması yaşamadan çalışabilmesi için Dockerfile ve docker-compose.yml dosyaları hazırlanmıştır. Sistem, başka bir cihaza klonlandığında docker-compose up komutuyla saniyeler içinde eksiksiz ayağa kalkabilmektedir.
- [Github Proje Linkimiz](#)

6. AI API'sine Gönderilen Prompt'ların Dökümü

- Proje geliştirme sürecinde, veri yapılarının sıfırdan (from scratch) tasarlanması, sentetik test verilerinin üretilmesi ve mikroservis mimarisinin ayağa kaldırılması aşamalarında GenAI araçlarından (ChatGPT, Gemini vb.) destek alınmıştır. Bu süreçte kullanılan temel mühendislik promptları aşağıda kategorize edilmiştir:
- 6.1. Veri Yapısı ve Algoritma Promptları
 - "Dijkstra algoritması için Min-Heap (priority queue) veri yapısı tasarla. Ekle, çıkar (extract-min) ve peek işlemleri olmalı. HeapifyUp ve HeapifyDown ile logaritmik zamanda işlem yapmalı. Python ile sıfırdan yaz."
 - "Open addressing (linear probing) ile çalışan bir Hash Table yaz. Hash fonksiyonu: anahtar % 1009 (C# backend ile uyumlu). Python ile sıfırdan yaz."
 - "2 boyutlu koordinat düzleminde en yakın K nokta aramasını verimli yapabilecek bir KD-Tree veri yapısı tasarla. Ortalama $O(\log N)$ arama karmaşıklığı hedefle."

- 6.2. Mimari ve Servis Promptları
 - "Python FastAPI ile bir AI mikroservisi tasarla. Endpointler: GET /duraklar, POST /rota-hesapla vb. Veriyi JSON dosyalarından yükle. CORS desteği ekle."
 - "20 duraklı bir toplu taşıma ağı için duraklar arası hat bağlantıları üret. Aynı iki durak arasında birden fazla hat olabilir (multigraph)."

(Not: Tüm prompt dökümünün tamamı projenin GitHub reposundaki ai_prompt_dokumu.md dosyasında mevcuttur.)